

"K-Means Clustering"

A unsupervised machine learning algorithm used to group similar data points into clusters based on features.

a group of similar data points that are more closely related to each other than to those in other groups.

A cluster is a collection of points

→ closed together (based on some distance metric like euclidean)

→ similar in features.

→ Belongs to the same group.

Centroid the center point of a cluster.

• It's like the avg. location of all points in a cluster.

• In K-Means every cluster has a centroid.

• It's not necessarily a real data point, but a calculated position (mean of all the points in the cluster)

$$\text{centroid} = \frac{1}{n} \sum_{i=1}^n x_i$$

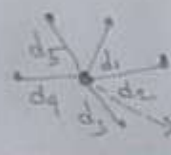
If n is the rows and we have d feature/dimensions then

$$x_i = [\text{age}, \text{income}, \text{score}]$$

$$\text{centroid } \mu = [\text{mean age}, \text{mean income}, \text{mean score}]$$

Steps

- ① Decide n clusters.
- ② Initialize n centroids for n clusters randomly.
- ③ assign all the points to a cluster based on shortest euclidean distance.
- ④ move centroid. So after step-3 re-calculate the centroid based the cluster we formed.



$$WSS = d_1^2 + d_2^2 + \dots + d_n^2$$

Repeat step-3 and step-4. If last calculated centroid and current calculated centroid is same then stop.

→ How to decide the number clusters?
using **Elbow** method

a technique which helps to find the optimal number of clusters (value of k)

Before running the algorithm we have to fix the value of k . Here comes the Elbow method.

→ How it works?

① Run k -means for a range of values of k .

② For each k calculate the within-cluster sum of squared errors (WSS) also known as **inertia**

$$WSS = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

k = number of clusters.
 C_i = all the data points in i -th cluster.
 μ_i → i -th clusters centroid.

③ Plot k v/s WSS

④ The curve will decrease sharply at first then level off.

The point where WSS stops decreasing significantly is known as **Elbow point**

WSS → measure of compactness.

Low WSS → data points are tightly packed within clusters.

$k \uparrow$ WSS $\downarrow$ because

more clusters → few points per cluster → points are closer to centroid.

But adding too clusters can lead to overfitting.

How to know WSS stopped Decreasing significantly?

There are 3 main approaches.

① Visual Inspection $\rightarrow$ by plotting K v/s WSS.

② Percentage Decrease Heuristic.

$\rightarrow$ find how much WSS has decreased to the previous.

$$\text{Relative Drop} = \frac{WSS_{k-1} - WSS_k}{WSS_{k-1}} \times 100\%$$

when $RD < 10\%$ means WSS isn't improving significantly any more.

③ Knee Detection Algorithm

$\Rightarrow$ It's work in 4 steps.

① Normalize:

The data point (x, y) here $x = k$ and $y = WSS$ are normalized so both axes are on the same scale (0 to 1)

② Line between endpoints: Draw a straight line between the 1st point (smallest k) and last point (largest k)

③ Calculate distance

• For each point on the curve, calculate the perpendicular distance to this straight line.

$$\text{distance} = \frac{|(B_x - A_x)(A_y - P_y) - (A_x - P_x)(B_y - A_y)|}{\sqrt{(B_x - A_x)^2 + (B_y - A_y)^2}}$$

④ Find the knee: The point with the maximum distance is the knee/elbow point, because that's where the curve bends most away from the straight line.

Perpendicular distance from point P to line AB

use cases of K-Mean clustering

① To segment data into groups.

→ Customer Segmentation (group customer based on buy, budget)

→ Market Segmentation: Identity different segments from survey data.

→ users behaviour clustering

② To Reduce Data Complexity.

→ Image Compression (group images of same person)

③ To find structure in high dimension data.

→ cluster gene expression

→ cluster text documents

Assumptions and when need to be careful:

works good when

- clusters are spherical
- all clusters are roughly equal size
- reasonable guess the number of clusters.